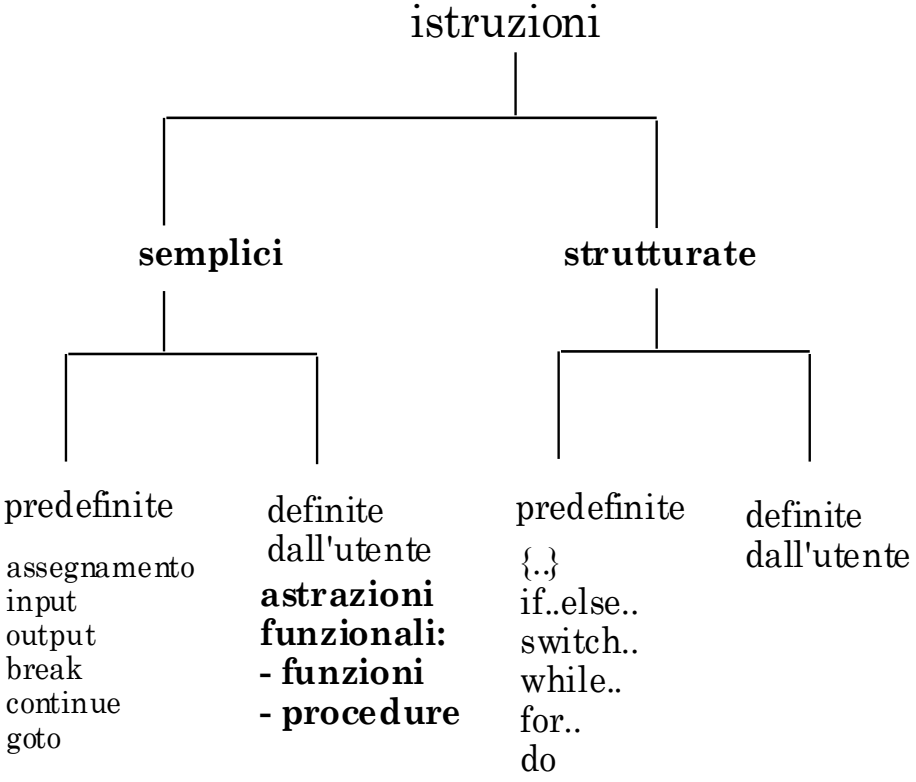
Astrazioni funzionali: Funzioni e Procedure



Spesso puo` essere utile avere la possibilita` di costruire delle nuove istruzioni che risolvano parti specifiche di un problema.

Ad esempio:

Ordinamento di un insieme:

```
#include <stdio.h>
#define dim 10

main()
{int V[dim], i,j, max, tmp, quanti;

/* lettura dei dati */
for (i=0; i<dim; i++)
  { printf("valore n. %d: ",i);
    scanf("%d", &V[i]);
   }
/*ordinamento */

for(i=0; i<dim; i++)
  {quanti=dim-i;
  max=quanti-1;
  for( j=0; j<quanti; j++)
    { if (V[j]>V[max])
         max=j;
    }
  if (max<quanti-1)
    { /*scambio */
    tmp=V[quanti-1];
    V[quanti-1]=V[max];
    V[max]=tmp;
    }
  }
/*stampa */
for(i=0; i<dim; i++)
  printf("Valore di V[%d]=%d\n", i, V[i]);
```

```
}
```

Potrebbe essere conveniente scrivere lo stesso algoritmo in modo piu` **astratto**:

```
#include <stdio.h>
#define dim 10

main()
{
int V[dim];

/* lettura dei dati */
leggi(V, dim);

/*ordinamento */
ordina(V, dim);

/*stampa */
stampa(V,dim);
}
```

☞ **leggi(),ordina(),stampa()** sono

astrazioni funzionali.

Astrazioni funzionali (*funzioni* e *procedure*):

- Consentono di definire nuove istruzioni che agiscono sui dati utilizzati dal programma, nascondendo la sequenza delle operazioni effettivamente eseguite dalla macchina.

- Vengono realizzate mediante la definizione di unita` di programma (***sottoprogrammi***) distinte dal programma principale (*main*) dando un nome ad un gruppo di istruzioni e stabilendo le modalita` di comunicazione tra l'unita` creata e quella in cui viene utilizzata.

Tutti i linguaggi di alto livello offrono la possibilita` di utilizzare funzioni e/o procedure:

- costrutti per la **definizione** di sottoprogrammi

- meccanismi per l'**utilizzo** di sottoprogrammi (meccanismi di ***chiamata***)

Funzioni e Procedure

Definizione:

Nella fase di **definizione** di un sottoprogramma (funzione o procedura) si stabilisce:

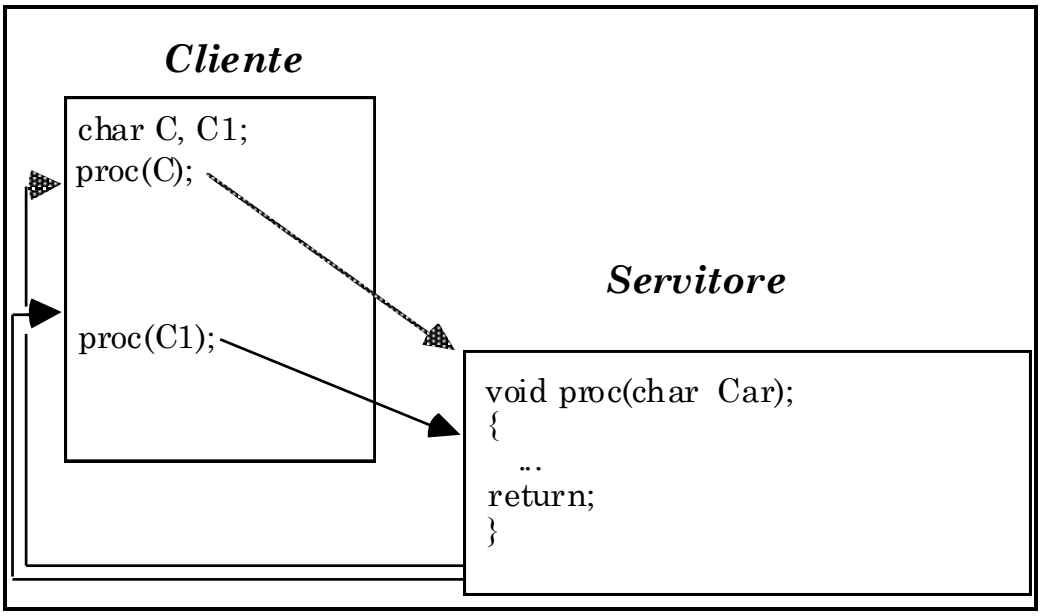
- un **identificatore** del sottoprogramma
- si esplicita il **codice** eseguito dal sottoprogramma (nel linguaggio utilizzato)
- si stabiliscono le **modalita` di comunicazione** tra l'unita` di programma che usa il sottoprogramma ed il sottoprogramma stesso (definizione dei **parametri formali**).

Utilizzo di funzioni/procedure:

- Per eseguire il gruppo di istruzioni del sottoprogramma, si utilizza l'identificatore assegnato al sottoprogramma in fase di definizione (***chiamata*** o invocazione del sottoprogramma).

- Alla chiamata viene sospesa l'esecuzione del programma (o dell'unita`) che contiene l'invocazione ed il controllo passa al sottoprogramma chiamato.

Modello ***Cliente-Servitore***



Parametri:

I ***parametri*** costituiscono il mezzo di comunicazione tra unita` chiamante ed unita` chiamata.

parametri formali: sono quelli specificati nella definizione del sottoprogramma (nell'esempio, **Car**). Sono in numero prefissato e ad ognuno di essi viene associato un tipo.

parametri attuali: sono i valori (o le variabili) effettivamente specificati all'atto della chiamata (nell'esempio, C e C1).

Parametri:

- Parametri ***attuali*** (specificati nella chiamata) e ***formali*** (specificati nella definizione) devono corrispondersi in ***numero***, ***posizione*** e ***tipo***.

- All'atto della chiamata avviene il ***legame dei parametri***, cioe` ai parametri formali vengono associati i parametri attuali. Esistono, in generale, varie forme di legame.

Esempio:

```
#include <stdio.h>

int f(int A, float B, char C)
{
/* definizione della funzione f*/
...
}


main()
{
  int p1;
  float p2;
  char p3;

  ...
  f(p1, p2, p3); /* corretto */
  f(p2, p1, p1); /* scorretto!*/
  f(p1, p2);       /* scorretto!*/
}
```

Funzioni e Procedure

Vantaggi:

- **riutilizzo di codice**: sintetizzando in un sottoprogramma un sotto-algoritmo, si ha la possibilita` di invocarlo piu` volte, sia nell'ambito dello stesso programma, che nell'ambito di programmi diversi (evitando di dover replicare ogni volta lo stesso codice).
- migliore **leggibilita`**: si ha in fatti una maggiore capacita` di astrazione
- sviluppo **top-down**: si delega a funzioni/procedure da sviluppare in una fase successiva la soluzione di sottoproblemi.

- testo del programma piu` **breve**: minore probabilita` di errori, dimensione del codice eseguibile piu` piccola.

☞ Utilizzando le astrazioni funzionali, la struttura dei programmi risulta articolata in **piu` unita`** (programma principale + sottoprogrammi).

Astrazioni funzionali

Si suddividono in procedure e funzioni:

Procedura:

E` un'astrazione della nozione di ***istruzione***. E` un'istruzione non primitiva attivabile in un qualunque punto del programma in cui puo` comparire un'istruzione.

Funzione:

E' l'astrazione del concetto di ***operatore*** (funzione o predicato) su un tipo di dato (primitivo o definito da utente). Si puo` attivare durante la valutazione di una qualunque espressione e **restituisce un valore.**

Ad esempio:

```
{
  leggi(N);/* procedura*/
  Fatt = fattoriale(N); /* funzione*/
};
```

☞ Formalmente, in C i sottoprogrammi sono soltanto ***funzioni***; le procedure possono essere realizzate come funzioni che non restituiscono alcun valore (**void**).

Funzioni in C

La **definizione** di procedure e funzioni segue regole sintattiche simili.

Definizione di funzione:

<intestazione>
{ **<parte-dichiarazioni> <parte-istruzioni>** }

Quindi, per definire una funzione, e` necessario specificare una ***intestazione*** e un ***blocco*** {..}:

Intestazione:

<tipo-ris> <nome> ([<lista-par-formali>])

L' intestazione contiene:

- **tipo del risultato** (***codominio***). Il tipo restituito puo` essere predefinito o definito dall'utente. Una funzione non puo` restituire valori di tipo:
  - **vettore**
  - **funzione**

- **identificatore** (*nome*) della funzione

- **lista dei parametri formali** (***dominio***). Per ciascun parametro formale viene specificato il tipo ed un identificatore che e` un nome simbolico per rappresentare il parametro all'interno della funzione (nel *blocco*). I parametri sono separati mediante virgola).

Definizione di Funzioni

Esempio:

```
int max (int a, int b) /*intestaz. */
{
  if (a>b) return a;
  else return b;
}
```

Blocco:

- Nella parte **blocco** possono essere presenti definizioni e/o dichiarazioni locali (***parte dichiarazioni***) e segue un **corpo** in cui viene specificato l'algoritmo rappresentato dalla funzione (***parte istruzioni***).
- I dati riferiti nel blocco possono essere **costanti**, **variabili**, oppure **parametri formali**.
- All'interno del blocco, i parametri formali vengono trattati come variabili.

Esempio:

```
#define N 100

typedef   char vettore[N];

int minimo (vettore vet)
{
int i, v, min;/* def. locali a minimo */
  for (min=vet[0], i=1; i<N; i++)
    {  v=vet[i];
       if (v<min) min=v;
    }
  return min;
}
```

☞ i, v, min sono ***variabili locali*** (dette variabili ***automatiche***):

- **tempo di vita**: esistono solo durante l'esecuzione della funzione minimo
- **visibilita`**: sono visibili (cioe` utilizzabili) soltanto all'interno della funzione minimo. *AL DI FUORI DELLA FUNZIONE LE VARIABILI LOCALI NON SONO UTILIZZABILI!!!!!*

Istruzione return:

return [<espressione>]

restituisce il controllo al chiamante e assegna all'identificatore della funzione il valore dell'<espressione> (se specificato).

Esempio:

```
int  read_int () /* intest. */
{
 int a
 scanf("%d", &a);
 return a;
}
```

Possono essere presenti ***piu` istruzioni return***:

```
int max (int a, int b) /*intest.*/
{
 if (a>b)    return a;
 else   return b;
}
```

o ***nessuna***:

```
void print_int (int a) /* intestazione */
{
 printf("%d", a);
}
```

Esempio:

```
/* funzione elevamento a potenza */

long power (int base, int n)
{
 int i;
 long p=1;

 for (i=1;i<=n;++i)
       p *= base; /* p = p*base */
 return p; /* ritorna il risultato */
}
```

Chiamata di funzioni:

All'interno di una espressione:

... NomeFunzione(Lista Parametri Attuali) ...

Esempio:

```
main()
{
int x=5;
printf("%d %d %d\n",
       x,power(x,2),power(x,3));
...
x=max(power(x,2), 30);
printf("%d\n", x);
}
```

Realizzazione Procedure in C

Una funzione puo' anche avere nessun valore (void) come risultato.

void	insieme vuoto di valori (dominio vuoto)
void fun(...)	funzione che non restituisce alcun valore
	› procedura

Esempio:

```
void print_int (int a) /* intestazione */
{
 printf("%d", a);
}
```

- ☞ In questo modo si realizza il concetto di **procedura**: non e` necessario prevedere l'istruzione di return all'interno del blocco; se si utilizza, **non si deve specificare alcun argomento**: **return;**
- ☞ Una funzione puo` anche non avere parametri. Alcune versioni di C esigono che si specifichi nell'intestazione *"void"*, altre accettano una sintassi del tipo:

```
void dummy()
{
printf("Ciao!\n");
 }
```

Esempio:

```
#include <stdio.h>

int max (int a, int b) /*def. max*/
{
  if (a>b) return a;
  else return b;
}

void print_int (int a) /* def. */
{
  printf("%d\", a);
  return;
}

void dummy() /*def. dummy */
{
printf("Ciao!\n");
}

main()
{
  int A, B;
  printf("Dammi A e B: ");
  scanf("%d %d", &A, &B);
  print_int(max(A,B));
  dummy();
}
```

Dichiarazione di funzione

Regola Generale:

Prima di utilizzare un identificatore e` necessario che sia gia` stato **almeno** dichiarato.

Funzioni C:

- **definizione**: descrive le proprieta` della funzione (tipo, nome, lista parametri formali) e la sua realizzazione (lista delle istruzioni coontenute nel blocco).
- **dichiarazione (*prototipo*)**: descrive le proprieta` della funzione senza definirne la realizzazione (***blocco***).

Dichiarazione di una funzione:

La **dichiarazione** di una funzione (***prototipo***) consiste nella sua intestazione, seguita da "**;**".
Ad esempio:

```
int max(int a, int b);
```

☞ Deve essere specificata ***prima*** della chiamata della funzione, **se la definizione segue (testualmente) la chiamata.**

Una funzione puo` essere ***dichiarata*** in punti diversi, ma e` ***definita*** una sola volta.

Esempio:

```
#include <stdio.h>

main()
{
  int A, B;
  printf("Dammi A e B: ");
  scanf("%d %d", &A, &B);
  printf("%d\n", max(A,B));
}

int max (int a, int b) /*intestaz. */
{
  if (a>b) return a;
  else return b;
}
```

☞ In questo caso il compilatore segnala un **errore** in corrispondenza della chiamata **max(A,B)**, perche` viene usato un identificatore che viene definito successivamente (dopo il main())

Soluzione:

```
#include <stdio.h>

int max(int a, int b);

main()
{
  int A, B;
  printf("Dammi A e B: ");
  scanf("%d %d", &A, &B);
  printf("%d\n", max(A,B));
}

int max (int a, int b) /*intestaz. */
{
  if (a>b) return a;
  else return b;
}
```

☞ La dichiarazione anticipa le proprieta` di un oggetto (la funzione) che verra` definito successivamente.

E le dichiarazioni di printf/scanf etc. ?

☞ sono contenute nel file stdio.h:

#include <stdio.h>

provoca l'inserimento del contenuto del file specificato.

Esempio:

Calcolo della radice intera di un numero intero letto a terminale.

```
#include <stdio.h>
/* prototipi delle funzioni: */

int   RadiceInt (int par);
int   Quadrato (int par);

main(void)
{ int X;
scanf("%d", &X);
printf("Radice: %d\n", RadiceInt(X));
printf("Quadrato: %d\n", Quadrato(X));
}

/* definizione funzioni: */

int   RadiceInt (int par) /* intest.*/
{int  cont = 0;
 while (cont*cont <= par)
   cont = cont + 1;
 return (cont-1); }

int   Quadrato (int par)/* intest. */
{       return (par*par); }
```

Protocollo da utilizzare:

<lista dichiarazioni di funzioni>
<main>
<definizioni delle funzioni dichiarate>

Tecniche di legame dei parametri

Al momento della chiamata di un sottoprogramma (funzione o procedura) viene effettuata l'associazione fra parametri attuali e parametri formali: **legame** (o **passaggio**) dei parametri.

In generale, esistono vari meccanismi di passaggio dei parametri.

Meccanismi piu` comuni:

- Legame per **valore** (C, Pascal);
- Legame per **indirizzo**, o per riferimento.

In C si utilizza il Legame per valore: il valore del parametro attuale viene copiato nel parametro formale

Legame per valore

All'atto della chiamata di una procedure/funzione P, per ogni parametro:

1. viene allocata una specifica cella di memoria associata per contenere il valore del parametro formale
2. vene valutato il valore del parametro attuale (puo` essere un'espressione), ed il suo valore viene **copiato** nella cella di memoria parametro formale

☞ Il parametro formale viene trattato come una **variabile locale** al sottoprogramma P: puo` essere modificato mediante assegnamento, etc.. In generale, al termine della chiamata, il parametro formale potra` assumere un valore diverso da quello iniziale.

Alla fine dell'esecuzione di P:

☞ **Se il parametro attuale e` una variabile**, al termine della chiamata, il suo valore rimane inalterato.

☞ **Se il parametro attuale e` una espressione**, il valore delle variabili che compaiono nell'espressione (nell'unita` di programma chiamante) rimane **inalterato**.

Non si vedono le modifiche fatte sul parametro formale perché queste sono state fatte su una cella di memoria diversa

Legame per valore

Quindi:

Parametri passati per valore servono a fornire **valori in ingresso** al sottoprogramma, ma non possono servire per modificare i valori nel programma chiamante.

☞ E` la tecnica di legame adottata dal C:

```
#include <stdio.h>

void P(int pf);

main()
{
int pa=10;

P(pa);
printf("valore finale di pa: %d\n",
   pa); /* pa vale sempre 10 */
}

void P(int pf)
{
pf=100;
printf("valore finale di pf: %d\n",
pf);
return;

}
```

Il fatto di avere modificato pf nella funzione non varie il valore di pa nel main!!!

Esempio: Funzione che scambia due variabili X, Y (integer) se X>Y e restituisce il valore minore tra i due.

```
#include <stdio.h>
int scambia1 (int A, int B);
main()
{ int X,Y ;
  scanf("%d %d", &X, &Y);
  printf("Scambia: %d %d %d\n",
          scambia1(X,Y), X,Y);
}

int scambia1 (int A, int B)
{ int   T;
  if (A>B)
    {  T=A;
       A=B;
       B=T;
       return A; }
  else   return B;
}
```

Dati in ingresso: 33 5 qual e` il risultato stampato?

Alla chiamata di scambia1:



Alla fine dell'attivazione di scambia1:

X	33	A	5
Y	5	B	33

NEL MAIN X e Y NON SONO STATI SCAMBIATI!!

Variabili Locali

Le variabili definite all'interno di una funzione/procedura non sono visibili e utilizzabili altro che all'interno di quella funzione o procedura.

ESEMPIO:

```
#include <stdio.h>

void funz()
{int h, i;

h=7; /* OK */
g=5; /* errore g è definita nel main e non
è utilizzabile qui */
i=10; /* OK ma attenzione la i definita qui
è diversa dalla i definita nel main */
}

void main()
{
int i, g;

g=7; /* OK*/
h=5; /* ERRORE: h è definita in funz e non
è utilizzabile nel main; */
i=10; /* OK ma attenzione la i definita qui
è diversa dalla i definita in funz */
}
```

Variabili GLOBALI

Quando c'è l'esigenza che più funzioni abbiamo accesso alle stesse variabili, queste variabili devono essere definite al di fuori di ogni procedura e funzione, e prima di esse

ESEMPIO:

```
#include <stdio.h>

int i; /* la variabile i è globale

void funz()
{int h;

h=7; /* OK */
g=5; /* errore g è definita nel main e non
è utilizzabile qui */
i=10; /* OK la i usata qui è la stessa che
usa il main */
}

void main()
{
int i, g;
g=7; /* OK*/
h=5; /* ERRORE: h è definita in funz e non
è utilizzabile nel main; */
i=10; /* OK la i usata qui è la stessa che
usa funz */
}
```

ESEMPIO: La funzione di scambio

```
#include <stdio.h>

int X, Y; /* variabili globali */

int scambia1 ();

main()
{ scanf("%d %d", &X, &Y);
  printf("Scambia: %d %d %d\n",
          scambia1(), X,Y);
}

int scambia1 ()
{ int   T;
  if (X>Y)
    {  T=X;
       X=Y;
       Y=T;
       return X;  }
  else   return Y;
}
```

Poiché in questo caso le variabili sono globali, la funzione scambia1 riesce effettivamente a invertire i valori.

Esercizio:

Programma che stampa i numeri primi compresi tra 1 ed n (n dato).

```
#include <stdio.h>
#include <math.h>

#define NO   0
#define YES  1

int isPrime(int n); /*dichiarazioni*/
int primes(int n);

void main()
{
int n;

  do {
       printf("\nNumeri primi non
              superiori a:\t");
       scanf("%d",&n);
     } while (n<1);

  printf("\nTrovati %d numeri primi.\n",
            primes(n));
}
```

```
int isPrime(int n)
{
int max,i;

if (n>0 && n<4) return YES;
      /* 1, 2 e 3 sono primi     */
else if (!(n%2)) return NO;
/* escludi i pari > 2   */
max = sqrt( (double)n );
/* CAST:sqrt ha arg double */
for(i=3; i<=max; i+=2)
  if (!(n%i)) return NO;
return YES;
}

int primes(int n)
{
  int i,count;

  if (n<=0) return -1;
  else count=0;

  if (n>=1)
      { printf("%5d\t",1);
        count++;
      }
  if (n>=2)
      { printf("%5d\t",2);
        count++; }
  for(i=3;i<=n;i+=2)
    if (isPrime(i))
        { printf("%5d\t",i);
          count++;
        }
  printf("\n");
```

```
    return count;
}
```

Vettori come parametri di funzioni

Per quanto riguarda i vettori, il C adotta una tecnica particolare per indicare i parametri formali:

```
int funz(int *v);
```

passa un vettore di interi come parametro, la dimensione non va indicata.

```
int funz(float *ff)
```

passa un vettore di float come parametro,

Una notazione simile si applica per le stringhe:

```
void funz3(char *, char *)
```

passa due stringhe come parametri

Vettori come parametri di funzioni

Per quanto riguarda i vettori, il C adotta una tecnica diversa nel passaggio dei parametri e prevede che parametri di tipo vettore debbano essere passati **attraverso il loro indirizzo**:

Ad esempio:

```
#include <stdio.h>
#define MAXDIM 30

int getvet(int *v, int maxdim);
/* notare che nei parametri formali di una
funzione il vettore si indica con la
notazione int *v */
main()
{
  int k, vet[MAXDIM];
  int dim;
  dim=getvet(vet, MAXDIM);
...
```

Definizione della funzione:

```
int getvet(int *v, int maxdim);
{ int i;
   for(i=0;i<maxdim;i++)
    {printf("%d elemento:\t", i+1);
      scanf("%d", &v[i]); }
  return n;}
```

☞ Il vettore e` modificato all'interno della funzione e le modifiche sopravvivono all'esecuzione della

funzione poiche` in C i vettori sono trasferiti ***per indirizzo***.

I Parametri del Main

Lo stesso main può essere considerato una procedura a cui si possono passare dei parametri. In particolare, i parametri che di possono passare al C rappresentati un vettore di stringhe.

Si indica così:

```
main(int arcg, char **argv)
```

argc indica il numero di parametri totali (è un parametro che non bisogna passare esplicitamente, ma è inserito dal sistema operativo automaticamente)
argv[1] primo parametro stringa
argv[2] secondo parametro stringa
e così via, per quante ce ne sono….

I parametri stringa sono passati all'invocazione del programma stesso:

```
nome_programma stringa1 stringa 2 …
```

ESEMPIO (supponiamo che il programma sotto si chiami "pippo.exe":

```
#include <stdio.h>



void main(int arcg, char **argv)
{
int i;

  printf("Valore di argc %d \n", argc);

  for(i=1; i<argc;io++){
   printf("Stringa %d = %s\n", i, argv[i]);
   }
}
```

Se invochiamo il programma così:

```
pippo addio mondo crudele
```

L'output del programma e':

```
Valore di argc 4
Stringa 1 = addio
Stringa 2 = mondo
Stringa 3 = crudele
```